

ATIVIDADE A1 DE MODELAGEM COMPUTACIONAL VALIDADOR DE LOGS E PROTOCOLOS

Juan Diego Bueno da Silva
Nathan Henrique Lucindo dos Santos
Paulo Alexandre Bezerra Haliski

INTRODUÇÃO

Este relatório técnico apresenta o desenvolvimento e a implementação prática de simuladores computacionais desenvolvidos em linguagem Python. O objetivo principal do projeto foi transpor os conceitos teóricos abordados ao longo do semestre letivo para o ambiente prático de programação, materializando validadores para três classes distintas da hierarquia de linguagens:

- **Linguagens Regulares:** Caracterizadas pela menor complexidade na hierarquia, validadas por meio da simulação de Autômatos Finitos.
- **Linguagens Livres de Contexto (LLC):** Que demandam controle de escopo e memória auxiliar, validadas através da simulação de Autômatos com Pilha.
- **Linguagens Recursivas:** Posicionadas no topo da hierarquia de decisões computáveis, validadas via emulação de Máquinas de Turing.

Nas seções seguintes, serão detalhados a modelagem matemática de cada linguagem escolhida, a arquitetura lógica dos simuladores em Python, os casos de testes aplicados e a análise dos resultados obtidos.

LINGUAGEM REGULAR

A linguagem definida aceita uma sequência de eventos que começa com LOGIN, seguido por AUTH, seguido por zero ou mais REQUEST 's, finalizando com LOGOUT. O modelo usado para representar a linguagem é o Autômato Finito Determinístico, ou DFA.

Possui alfabeto definido por:

$\Sigma = \{\text{LOGIN, AUTH, REQUEST, LOGOUT}\}$

Pode ser definido pela definição formal:

$L1 = \{ \text{LOGIN AUTH REQUEST}^* \text{ LOGOUT} \}$

ou

$L1 = \{ w \in \Sigma^* \mid w = \text{LOGIN AUTH REQUEST}^k \text{ LOGOUT}, k \geq 0 \}$

Cadeias aceitas pelo modelo:

LOGIN AUTH LOGOUT;

LOGIN AUTH REQUEST LOGOUT;

LOGIN AUTH REQUEST REQUEST LOGOUT

Cadeias rejeitadas pelo modelo:

LOGIN AUTH REQUEST;

LOGIN AUTH REQUEST REQUEST;

LOGIN AUTH LOGOUT REQUEST LOGOUT

LINGUAGEM LIVRE DE CONTEXTO

A linguagem aceita blocos de transação que abrem com BEGIN e fecham com END, podendo estar aninhados. O modelo utilizado para representar a linguagem é o Autômato de Pilha, PDA.

Possui o alfabeto definido por:

$\Sigma = \{ \text{BEGIN}, \text{END} \}$

Pode ser definido pela definição formal:

$V = \{ S \} \quad \Sigma = \{ \text{BEGIN}, \text{END} \} \quad R = \{ S \rightarrow \text{BEGIN } S \text{ END} \mid \text{BEGIN END} \}$

Cadeias aceitas pelo modelo:

BEGIN END;

BEGIN BEGIN END END;

BEGIN BEGIN BEGIN END END END

Cadeias rejeitadas pelo modelo:

BEGIN END END;

BEGIN BEGIN END;

BEGIN END BEGIN

LINGUAGEM RECURSIVA

A linguagem aceita um trio balanceado de eventos, sendo OPEN, COMMIT e CLOSE, respectivamente, com pelo menos 1 registro de cada. O modelo que representa o autômato é a Máquina de Turing.

Possui o alfabeto definido por:

$$\Sigma = \{\text{OPEN}, \text{COMMIT}, \text{CLOSE}\}$$

Pode ser definido pela definição formal:

$$L3 = \{ \text{OPEN}^n \text{ COMMIT}^n \text{ CLOSE}^n \mid n \geq 1 \}$$

ou

$$L3 = \{ w \in \Sigma^* \mid \exists n \geq 1 : w = \text{OPEN}^n \text{ COMMIT}^n \text{ CLOSE}^n \}$$

DIAGRAMAS E TABELAS DE TRANSIÇÃO

LINGUAGEM REGULAR

Diagrama:

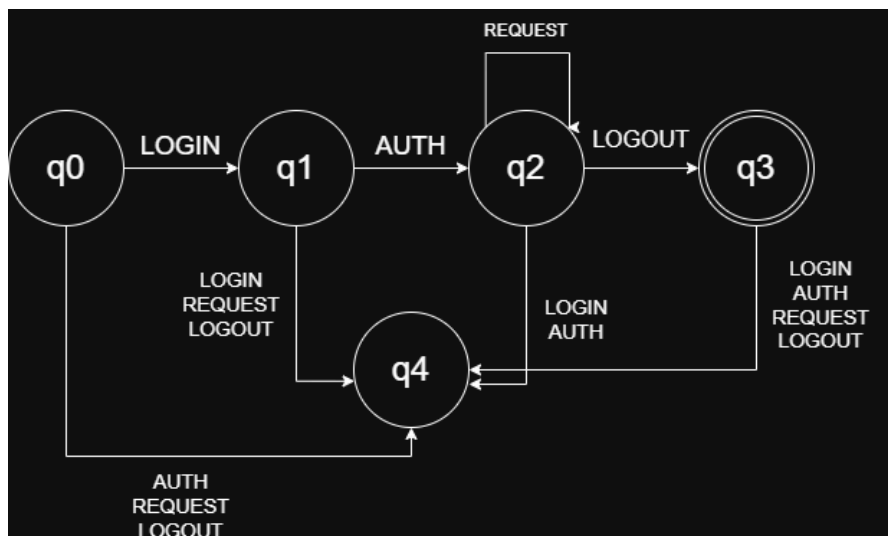


Tabela de transição:

δ	LOGIN	AUTH	REQUEST	LOGOUT
$\rightarrow$ q0 (inicial)	q1	q4	q4	q4

q1 (após LOGIN)	q4	q2	q4	q4
q2 (após AUTH)	q4	q4	q2	q3
* q3 (após LOGOUT)	q4	q4	q4	q4
q4 (morto)	q4	q4	q4	q4

LINGUAGEM LIVRE DE CONTEXTO

Diagrama:

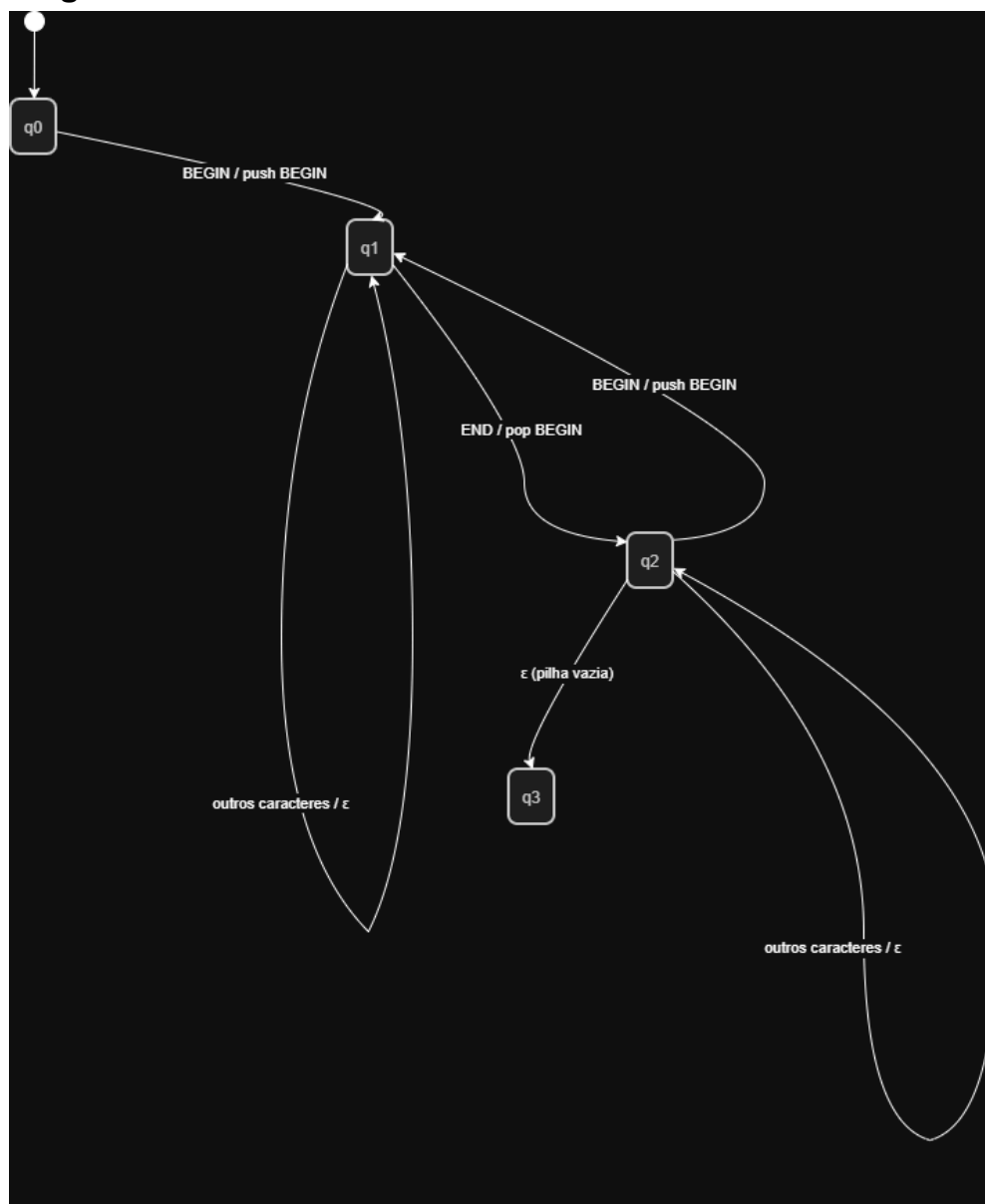


Tabela de transições:

Estado	Símbolo	Topo da pilha	Novo estado	Ação na pilha
$\rightarrow$ * q_lendo	BEGIN	$\emptyset$ (vazia)	q_lendo	empilha B
q_lendo	BEGIN	B	q_lendo	empilha B
q_lendo	END	B	q_lendo	desempilha B
q_lendo	END	$\emptyset$ (vazia)	q_rej	- (nenhuma)

LINGUAGEM RECURSIVA

Diagrama:

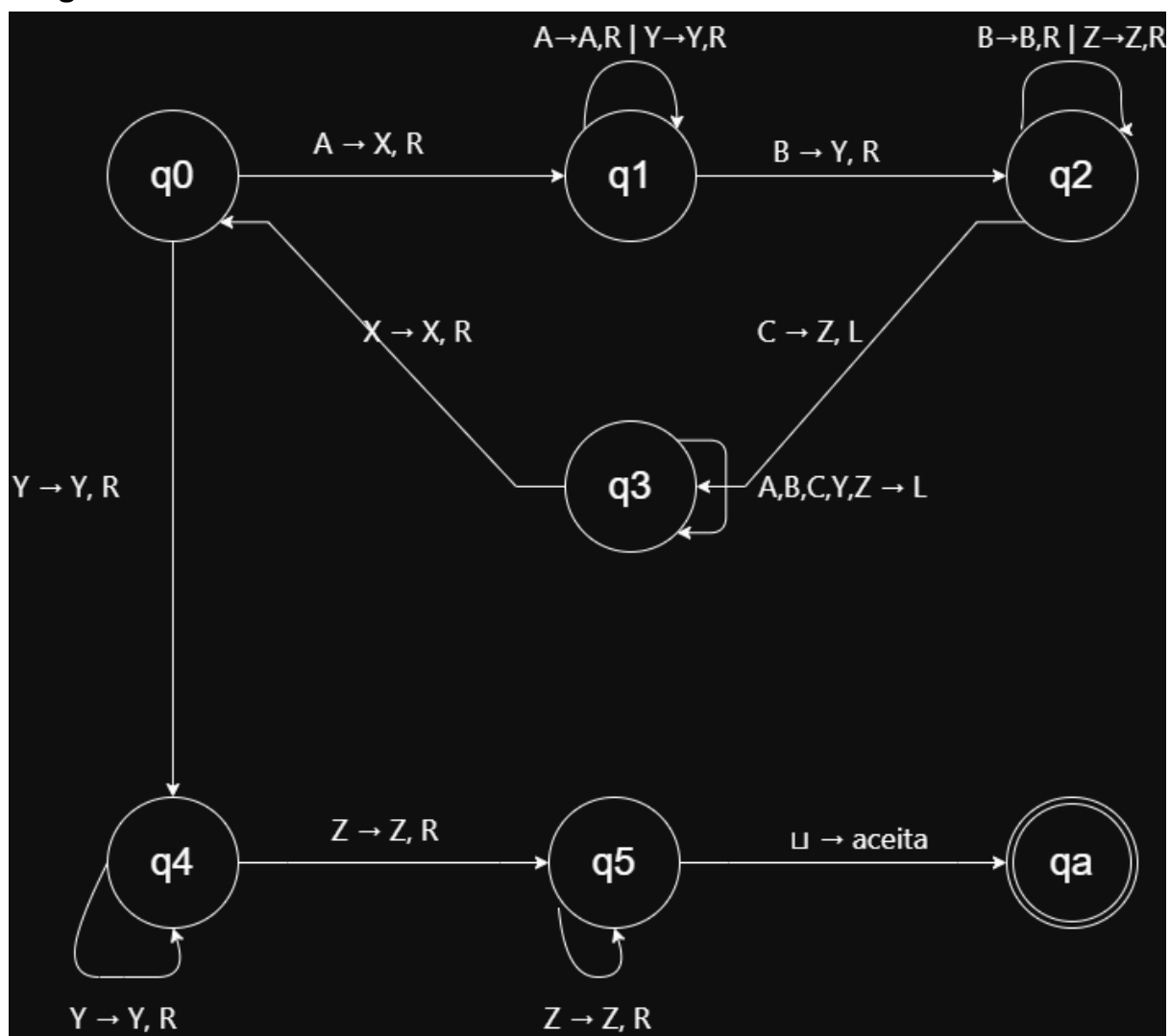


Tabela de transição:

Estado	Lê	Escreve	Move	Vai para	Propósito
→ q0	X	X	D	q0	pula OPENs já marcados
q0	OPEN	X	D	q1	marca um OPEN
q0	Y	Y	D	q4	acabaram os OPEN → verificar
q1	OPEN	OPEN	D	q1	avança até o COMMIT
q1	Y	Y	D	q1	avança até o COMMIT
q1	COMMIT	Y	D	q2	marca um COMMIT
q2	COMMIT	COMMIT	D	q2	avança até o CLOSE
q2	Z	Z	D	q2	avança até o CLOSE
q2	CLOSE	Z	E	q3	marca um CLOSE e rebobina
q3	OPEN	OPEN	E	q3	rebobina
q3	COMMIT	COMMIT	E	q3	rebobina
q3	Y	Y	E	q3	rebobina
q3	Z	Z	E	q3	rebobina
q3	X	X	D	q0	achou o X → nova passada
q4	Y	Y	D	q4	verificação final
q4	Z	Z	D	q4	verificação final
q4	_ (vazio)	_	D	* q_aceita	tudo balanceado → aceita

IMPLEMENTAÇÃO

O projeto está organizado em três módulos independentes, responsáveis pela simulação de um autômato distinto, mais um módulo de testes que orquestra a execução dos três casos de teste a partir dos arquivos na pasta testes/.

Nos três simuladores a tabela de transições é representada como um dictionary de Python cuja chave é a tupla (entrada, estado) e cujo valor é a tupla (novo estado, ...). Essa escolha permite escrever a função de transição em uma única estrutura, sem if/elif aninhados, e torna trivial conferir o autômato implementado contra o diagrama.

TESTES E RESULTADOS

DFA - Linguagem Regular

#	Entrada	Esperado	Obtido	Passos
1	LOGIN AUTH LOGOUT	Aceita	Aceita	3
2	LOGIN AUTH REQUEST LOGOUT	Aceita	Aceita	4
3	LOGIN AUTH REQUEST REQUEST LOGOUT	Aceita	Aceita	5
4	LOGIN AUTH REQUEST	Não aceita	Não aceita	3
5	LOGIN AUTH REQUEST REQUEST	Não aceita	Não aceita	4
6	LOGIN AUTH LOGOUT REQUEST LOGOUT	Não aceita	Não aceita	4

PDA - Linguagem Livre de Contexto

#	Entrada	Esperado	Obtido	Passos
1	BEGIN END	Aceita	Aceita	2
2	BEGIN BEGIN END END	Aceita	Aceita	4
3	BEGIN BEGIN BEGIN END END END	Aceita	Aceita	6
4	BEGIN END END	Não aceita	Não aceita	3
5	BEGIN BEGIN END	Não aceita	Não aceita	3
6	BEGIN END BEGIN	Não aceita	Não aceita	3

7.3 Máquina de Turing - Linguagem Recursiva

#	Entrada	Esperado	Obtido	Passos
---	---------	----------	--------	--------

1	OPEN COMMIT CLOSE	Aceita	Aceita	8
2	OPEN OPEN COMMIT COMMIT CLOSE CLOSE	Aceita	Aceita	~22
3	OPEN^3 COMMIT^3 CLOSE^3	Aceita	Aceita	~42
4	OPEN OPEN COMMIT CLOSE CLOSE	Não aceita	Não aceita	~6
5	OPEN COMMIT COMMIT CLOSE	Não aceita	Não aceita	~7
6	OPEN^3 COMMIT^2 CLOSE^3	Não aceita	Não aceita	~14